

Scheduling: Proportional Share

CS232 Operating Systems

Fall 2019

Taj Muhammad Khan

Proportional Share Scheduler

Basic Idea:

- Instead of optimizing for Turnaround Time or Response Time, the scheduler should instead guarantee that each job gets a certain percentage of CPU time.

For example CPU should be shared between A (75%) and B (25%)

Lottery scheduling

- Each process is awarded a certain number of ***tickets***.
- The percentage of tickets a process has represents its share of the resource in question
- Example: Total tickets = 100. A has 75 (0-74), and B has 25 (75-99)
- On every context switch the scheduler picks a winning ticket

Here is an example output of a lottery scheduler's winning tickets:

63 85 70 39 76 17 29 41 36 39 10 99 68 83 63 62 43 0 49

Here is the resulting schedule:

A A A A A A A A A A A A A A A
 B B B B

Tickets mechanisms

- Ticket currency

User A -> 500 (A's currency) to A1 -> 50 (global currency)
 -> 500 (A's currency) to A2 -> 50 (global currency)
User B -> 10 (B's currency) to B1 -> 100 (global currency)

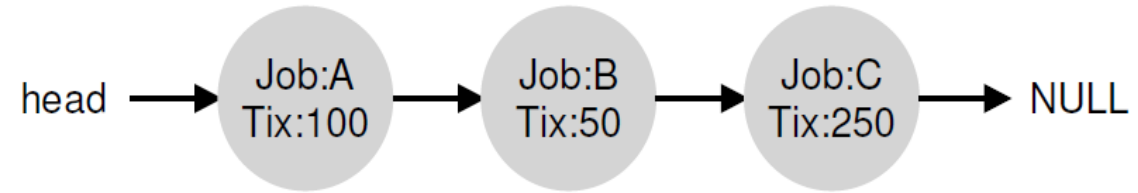
- Ticket transfer

- Give their tickets to somebody else (client/server example)

- Ticket inflation

- Works between trusting processes

Lottery: Implementation



```
1 // counter: used to track if we've found the winner yet
2 int counter = 0;
3
4 // winner: use some call to a random number generator to
5 //           get a value, between 0 and the total # of tickets
6 int winner = getrandom(0, totaltickets);
7
8 // current: use this to walk through the list of jobs
9 node_t *current = head;
10 while (current) {
11     counter = counter + current->tickets;
12     if (counter > winner)
13         break; // found the winner
14     current = current->next;
15 }
16 // 'current' is the winner: schedule it...
```

Lottery: Fairness

- Randomization introduces variance
- Over time it'd attain fairness
- Example
 - Two jobs
 - Both have 100 tickets
 - (Un)Fairness metric: both should finish at same time i.e.

$$U = \frac{\text{Finish_time}_A}{\text{Finish_time}_B}$$

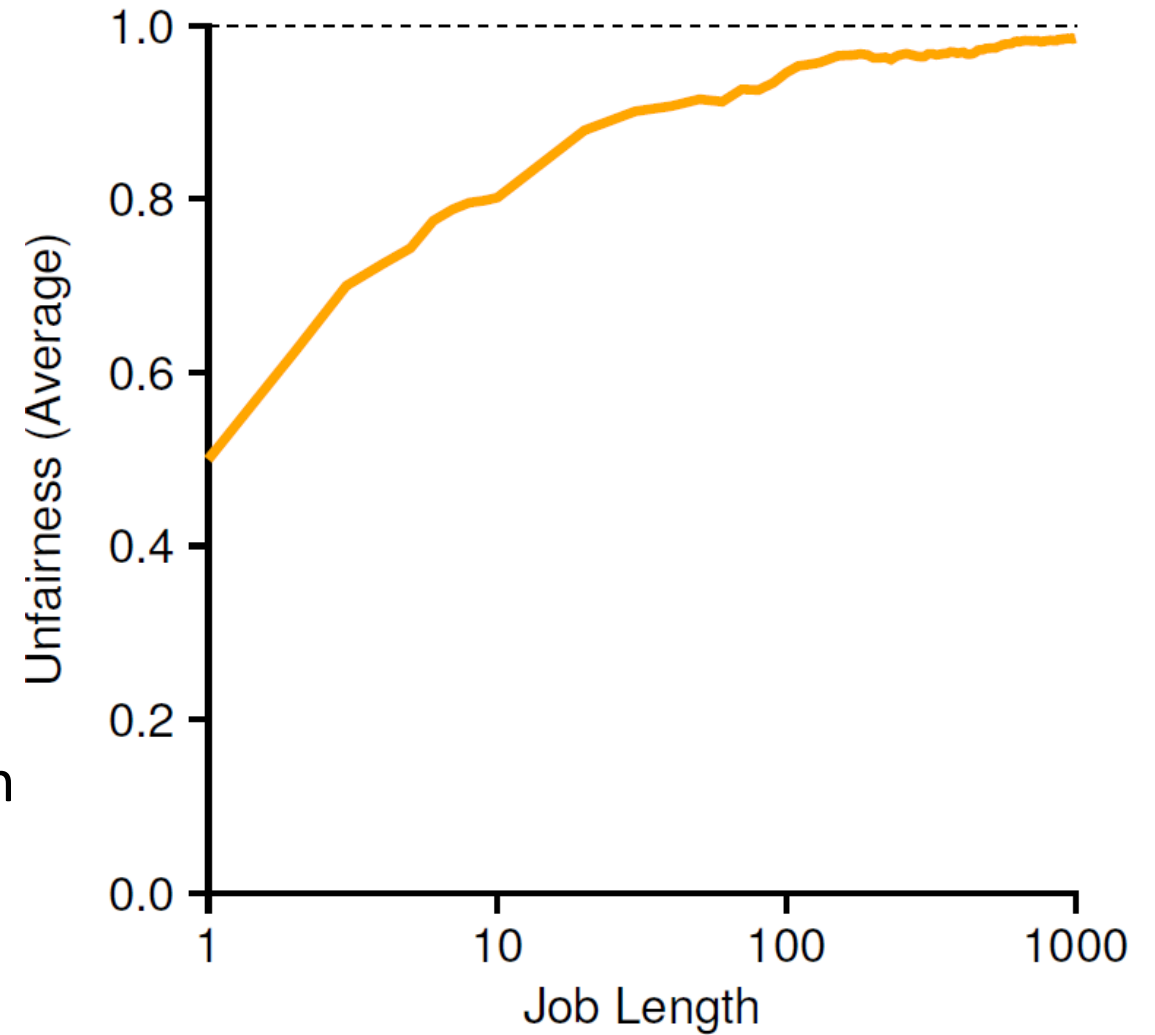


Figure 9.2: Lottery Fairness Study

Lottery: How to assign tickets?

Deterministic Proportional Scheduling

- Stride Scheduling

- $Stride = \frac{1}{Ticket_Share}$

- Each process has a counter (*pass* value)

- At any given time scheduler will:

- Select the process with minimum pass value
 - Increment its pass value by its stride
 - Schedule it

- E.g., A,B,C have 100,50,250 tickets

```
curr = remove_min(queue);  
schedule(curr);  
curr->pass += curr->stride;  
insert(queue, curr);
```

Deterministic Proportional Scheduling

Pass(A) (stride=100)	Pass(B) (stride=200)	Pass(C) (stride=40)	Who Runs?
0	0	0	A
100	0	0	B
100	200	0	C
100	200	40	C
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	...

Figure 9.3: **Stride Scheduling: A Trace**

Completely Fair Scheduler (CFS)

- Linux Scheduler
- Uses Fair Share Scheduling
- Doesn't have a fixed time slice
- Keeps a counter vruntime for each process
- When a scheduling decision occurs, it'd select the process with lowest vruntime to run
- Run for how long?
 - Time slice too small → good fairness but scheduling overhead
 - Time slice too big → low scheduling overhead but costs fairness in short term

CFS

- Maintains various parameters
 - `sched_latency` = 48
 - $time_slice = \frac{sched_latency}{num_processes}$
 - `time_slice` = 12, `num_processes`=4

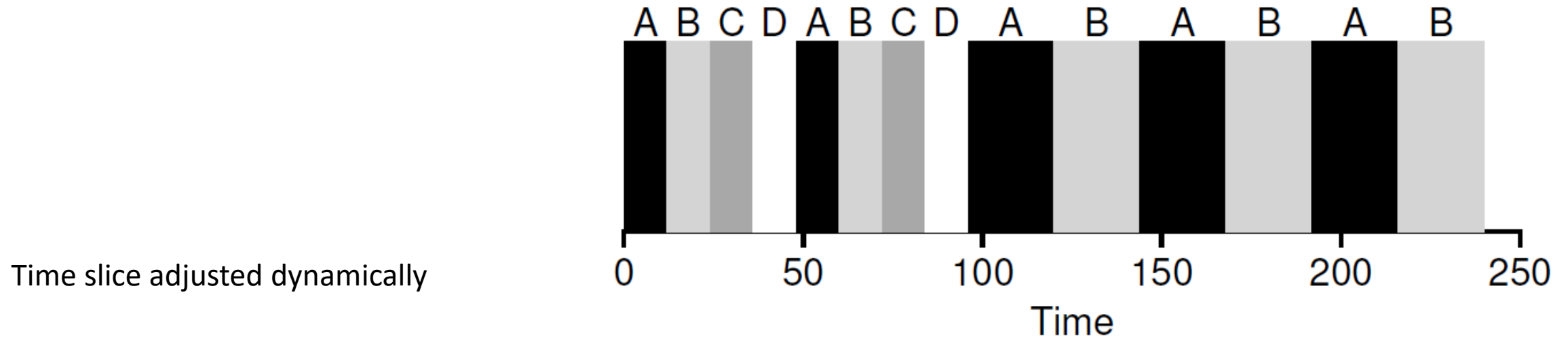


Figure 9.4: **CFS Simple Example**

CFS

- What if too many processes running?
 - min_granularity
 - $\text{time_slice} = \max(\text{min_granularity}, \text{calculated_time_slice_value})$
-
- timer interrupt goes off every 1 ms
 - everytime a process leaves CPU, its vruntime gets updated
 - this way eventually fairness is ensured

CFS: Niceness

- Processes can be nice to other processes (or not)
- A higher nice value would give them a lower weight → smaller slice
- nice_A = -5, nice_B = 0 (default value)

$$\text{time_slice}_k = \frac{\text{weight}_k}{\sum_{i=0}^{n-1} \text{weight}_i} \cdot \text{sched_latency} \quad (9.1)$$

- process vruntime will also increase in inverse proportion to its weight

$$\text{vruntime}_i = \text{vruntime}_i + \frac{\text{weight}_0}{\text{weight}_i} \cdot \text{runtime}_i \quad (9.2)$$

CFS: Nice values

```
static const int prio_to_weight[40] = {  
    /* -20 */ 88761, 71755, 56483, 46273, 36291,  
    /* -15 */ 29154, 23254, 18705, 14949, 11916,  
    /* -10 */ 9548, 7620, 6100, 4904, 3906,  
    /* -5 */ 3121, 2501, 1991, 1586, 1277,  
    /* 0 */ 1024, 820, 655, 526, 423,  
    /* 5 */ 335, 272, 215, 172, 137,  
    /* 10 */ 110, 87, 70, 56, 45,  
    /* 15 */ 36, 29, 23, 18, 15,  
};
```

Data structure: RB Tree

- Self balancing
 - Sorted
 - Removing a process is trivial
 - Insertion is $\log(n)$
 - Only has ready processes
-
- I/O, sleeping processes assigned min vruntime when they join.

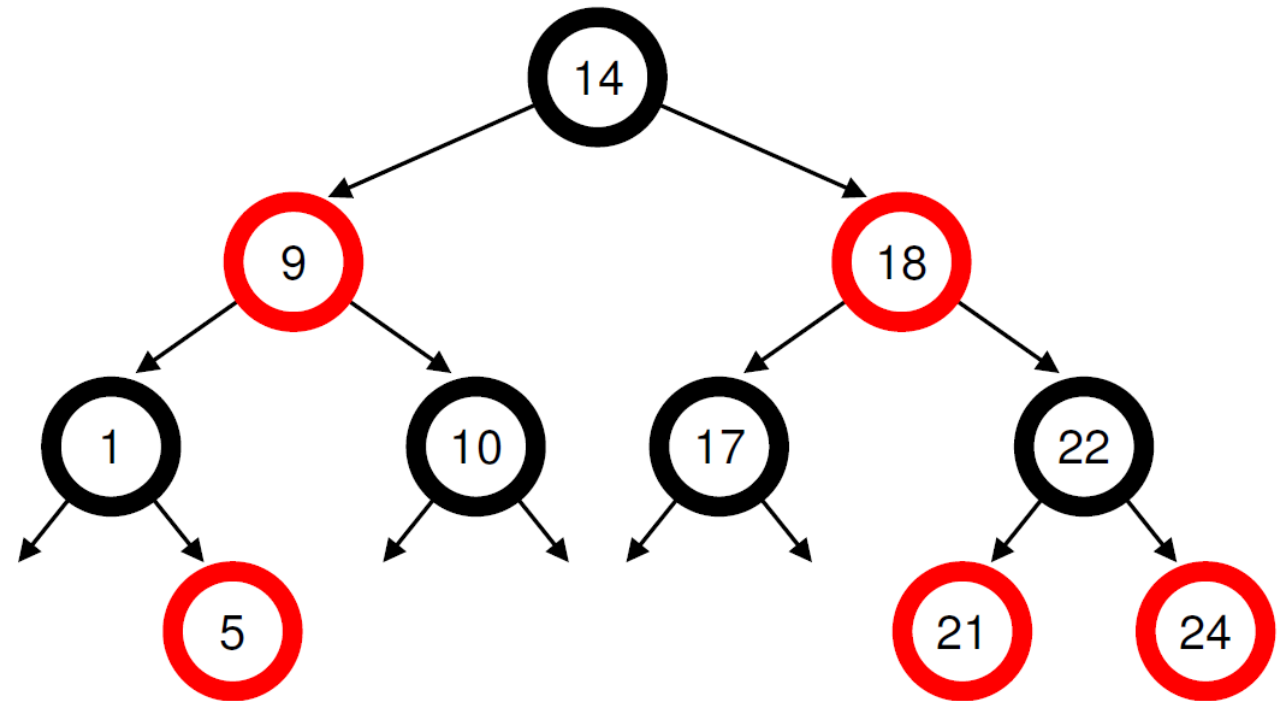


Figure 9.5: CFS Red-Black Tree